

ReFiles: A Capability-Oriented Architecture for Responsive File Browsing Across Heterogeneous Storage Providers

Tomoyuki Terashita

Files Community
onein528@gmail.com

Abstract

1 Introduction

1.1 The File Manager as a Systems Problem

A graphical file manager appears simple because its most visible operations are familiar: list a directory, open an item, copy or move it, and show a preview. Underneath that interface, however, a modern file manager coordinates several systems with different performance characteristics, failure modes, and ownership rules. Local files may be exposed through direct filesystem APIs or through the Windows Shell namespace. Archives behave like folders while requiring a decoding backend and, in some cases, credentials. Remote providers such as FTP add latency, partial failure, and session lifetimes. Property handlers, thumbnail providers, preview handlers, and change notifications introduce further platform-specific behavior. The user nevertheless expects all of these sources to appear within one coherent application.

The difficulty is not only functional coverage. A file manager is an interactive system, and its architecture is visible through latency. A directory may eventually enumerate correctly while still feeling unresponsive if the first useful row is delayed, if metadata work monopolizes the UI thread, or if thousands of fine-grained notifications overwhelm presentation. Conversely, aggressively parallel work may return after navigation has already changed, allowing stale properties or thumbnails to appear in a new view. Correctness therefore includes both the final result and the timing and lifetime boundaries through which that result travels.

ReFiles is an open-source desktop file manager implemented in C# and .NET with a WinUI 3 presentation layer. It is developed as an alternative exploration of the architecture behind a Windows file manager rather than as a thin visual wrapper around one storage API. Its design separates UI-independent application and storage state from presentation, represents provider-specific optional behavior through capabilities, publishes browse results progressively, and isolates platform work behind explicit scheduling and ownership boundaries.

This thesis uses ReFiles as a concrete systems-engineering case study. The objective is not to claim that the individual ideas of capability discovery, asynchronous enumeration, or layered architecture are new in isolation. Instead, it examines how these ideas can be combined into a practical file manager whose generic browsing code works across heterogeneous providers while remaining responsive and explicit about resource ownership.

1.2 Architectural Challenges

Three challenges shape the design: heterogeneous provider behavior, progressive presentation at scale, and the lifetime of asynchronous and platform-bound resources.

1.2.1 Heterogeneous storage providers

Storage providers share a small semantic core. They expose items with identities and names, resolve references, and usually offer some form of hierarchy or enumeration. Their optional behavior differs substantially. A local filesystem item may support efficient change notifications and Windows Shell properties. An archive entry may provide a readable stream but no native rename operation. An FTP item may expose remote metadata while lacking a thumbnail implementation. A virtual Shell item may support preview through a registered handler even when it does not map cleanly to a filesystem path.

A single universal interface tends to make these differences implicit. Optional members either throw when a provider does not implement them or require capability flags that must be kept consistent with the methods they guard. Provider type checks create a different problem: generic browsing and presentation code gradually learns about every backend, increasing coupling and making each new provider a cross-cutting change.

ReFiles instead treats optional behavior as a set of capabilities attached to an item. Generic consumers request the semantic behavior they need, such as property retrieval, thumbnail production, preview, stream access, or change observation. Absence is a valid result. A capability may be supplied directly by a core model, created by a provider-specific factory, composed from multiple candidates, or wrapped with cross-cutting behavior such as caching. This approach preserves a small required storage contract while allowing provider differences to remain behind UI-independent interfaces.

1.2.2 Latency and progressive feedback

Directory browsing is a pipeline rather than one operation. A location is resolved, items are enumerated, storage models are projected into browse state, changes are published to presentation, UI models are created or updated, and visual containers are realized. Properties and thumbnails enrich those rows later. Treating the pipeline as a single completion event hides the distinction between throughput and responsiveness.

ReFiles therefore considers the first realized row a more meaningful interactive milestone than the completion of enumeration. Item identity and basic display state can be published before expensive metadata. UI-thread work is issued in bounded batches so that the dispatcher can continue processing input and layout. Presentation keeps stable model identities when enrichment arrives instead of replacing every row. Virtualization limits the number of visual containers even when the logical result contains tens of thousands of items.

Progressive work also creates races. Navigation, refresh, filtering, or item replacement can invalidate an in-flight operation. Cancellation is useful but insufficient because a provider or platform handler may complete after cancellation was requested. The receiving layer must additionally verify that the result still belongs to the active browse generation and the same item identity before publishing it.

1.2.3 Ownership across asynchronous and platform boundaries

The file-manager domain contains many resources whose lifetime is not captured by ordinary managed references: streams, remote sessions, archive backends, cancellation registrations, Windows Shell objects, native buffers, preview handlers, and apartment-bound workers. A resource may be borrowed, shared, owned by one item, or tied to a broader provider session. Some resources require asynchronous cleanup, while others must be released on a particular thread or through a matching native allocator.

Implicit ownership makes failure paths especially dangerous. Capability creation can fail after earlier instances have already been created. A wrapper can fail while its inner object still needs disposal. Navigation can abandon work that later returns an owned result. Shutdown may need to aggregate cleanup failures without skipping remaining resources.

ReFiles addresses these cases by making ownership part of the architecture. Item-scoped capability instances are tracked and disposed in reverse construction order. Shared instances remain the responsibility of their composition root. Models support asynchronous disposal, and platform integration is kept behind schedulers and wrappers that preserve thread, allocator, and ownership requirements. Long-running storage operations may cross an out-of-process boundary rather than sharing UI state directly.

1.3 Design Position

The central design position of this thesis is that responsive, provider-agnostic browsing requires three boundaries to align:

1. **A semantic boundary:** generic code depends on item capabilities rather than provider identity.
2. **A publication boundary:** basic browse state is published progressively, while optional enrichment is scheduled and validated independently.

3. **An ownership boundary:** every resource has an explicit owner and cleanup path, including during cancellation, stale completion, partial construction, and application shutdown.

These boundaries reinforce one another. Lazy capabilities avoid constructing optional provider objects for every enumerated item. Progressive publication allows the UI to display identity before requesting enrichment. Explicit ownership lets delayed or rejected enrichment be cleaned up safely. Provider isolation, in turn, keeps platform-specific latency and failure from leaking into generic presentation semantics.

The architecture is divided into a UI-independent core, provider and platform implementations, presentation adapters, reusable controls, and an isolated operation host. The core owns storage references, stable identities, browse state, capability composition, session state, and operation intent. Presentation owns WinUI objects, dispatcher-bound publication, localized display state, viewport information, and conversion of encoded Core results into visual resources. This dependency direction allows the same storage and browsing semantics to be tested without constructing a WinUI visual tree.

1.4 Research Questions

The thesis is organized around the following research questions:

- RQ1** How can optional item behavior from heterogeneous storage providers be composed without expanding a monolithic storage interface or introducing provider-specific branches into generic code?
- RQ2** Which pipeline boundaries and measurements are needed to preserve interactive responsiveness while browsing and enriching directories containing tens of thousands of items?
- RQ3** How can cancellation, stale-result rejection, and explicit ownership be combined to make asynchronous browsing and Windows platform integration predictable under failure and navigation races?

The questions connect structural and empirical concerns. RQ1 is evaluated primarily through the capability model and provider case studies. RQ2 motivates deterministic browse scenarios, architecture benchmarks, and measurements at the actual WinUI row-realization boundary. RQ3 is examined through lifetime contracts, fault-oriented tests, and scenarios in which work completes after its originating state has become stale.

1.5 Contributions

This thesis makes four practical contributions:

1. It presents a capability-oriented item model in which factories, deterministic combinators, wrappers, lazy resolution, caching, and lifetime ownership form one composition mechanism for heterogeneous providers.
2. It describes a progressive browse and presentation pipeline that separates resolution, enumeration, projection, bounded UI publication, visual realization, and optional enrichment.
3. It develops an explicit lifetime model for managed, asynchronous, remote, and Windows Shell resources, including cleanup after partial capability construction and rejection of stale asynchronous results.
4. It defines an evaluation method that distinguishes deterministic architecture overhead from machine-dependent Shell and filesystem behavior, with milestones including first Core batch, first presentation item, first realized row, dispatcher latency, notification count, model churn, and virtualization.

The implementation supports Windows filesystem and Shell items, archive browsing, and FTP/FTPS sources through the same core model. These providers serve as vertical case studies: they vary in latency, identity, stream ownership, metadata availability, threading requirements, and supported operations, yet reuse the generic browse, property, thumbnail, preview, and presentation paths where the corresponding capabilities are available.

1.6 Evaluation Strategy

The evaluation separates controlled architecture measurements from end-to-end platform scenarios. Isolated benchmarks measure cold and cached capability resolution, thumbnail-cache behavior, and browse enumeration/projection/publication at representative sizes. Deterministic WinUI scenarios exercise the production path from a synthetic provider through browse state, presentation adapters, view models, collection views, and an actually realized table row. Current scenarios cover 100, 1,000, 10,000, and 44,000 items, including progressive property enrichment.

The primary interactive metric is time to first realized row. Supporting metrics include time to the first Core batch, time to the first presentation item, enumeration completion, maximum and 95th-percentile dispatcher latency, stalls

above several thresholds, collection notification count, unique view-model count, and maximum realized container count. Structural assertions verify correct final counts, bounded publication, stable model identity, and continued virtualization.

Real Windows folder scenarios are treated separately because they include disk and filesystem cache state, installed Shell extensions, property and thumbnail handlers, COM scheduling, and machine-specific behavior. Their results record environment metadata and are interpreted as scenarios rather than deterministic benchmark scores. This separation avoids presenting noisy hosted-runner timings as universal performance claims.

Comparative evaluation should emphasize architectural ablations before comparisons with other file managers. Useful baselines include progressive versus completion-gated publication, bounded batches versus one dispatcher callback per item, lazy versus eager capability creation, and cold versus cached capability resolution. These comparisons can isolate the effects of specific design choices using the same codebase and instrumentation.

1.7 Scope and Limitations

ReFiles is a Windows desktop application, and several case studies depend on Windows Shell and WinUI behavior. The architecture can express non-Windows providers, but this thesis does not claim platform-independent UI portability. FTP and archive support demonstrate provider diversity without representing every cloud, distributed, or content-addressed storage model.

The work also does not assume that architecture alone guarantees performance. Batch sizes, concurrency limits, caching policies, handler behavior, and UI layout remain empirical concerns. Timing measurements from real folders are sensitive to hardware, cache state, operating-system version, and installed extensions. The thesis therefore distinguishes stable structural invariants from environment-dependent measurements and avoids direct performance claims against other file managers unless equivalent instrumentation is available.

Finally, capability composition reduces provider coupling but does not eliminate semantic design work. Each capability still requires a clear contract for absence, cancellation, concurrency, result ownership, priority, and composition. The approach moves these decisions into explicit boundaries; it does not make them automatic.

1.8 Thesis Overview

The remainder of this thesis develops the capability model, progressive browsing pipeline, presentation boundary, and ownership rules in detail. It then describes the Windows, archive, and FTP implementations as case studies, evaluates the architecture through controlled and end-to-end measurements, and concludes with limitations and directions for future provider, search, operation, and observability work.